Exercise Sheet 2: Backpropagation and Feedback Alignment

Ella Capellini — ecapellini@student.ethz.ch

Introduction

Backpropagation (BP) sends the error backwards through the network with the transpose of the forward weights, $\delta_h = W^T \delta_{out}$. In feedback alignment (FA) this W^T is replaced by a fixed random matrix B, so $\delta_h = B\,\delta_{out}$. This looks like it should not work, because B has nothing to do with the task. *why does a random B still give a useful learning signal?* The answer will be that the forward weights W adapt during training, so that W^T becomes similar to B. I measure this with the angle between B and W^T. In the three experiments below I first check whether this alignment happens (linear network), then whether it still happens with a nonlinearity and what it costs compared to BP, and finally what kind of solution BP and FA find in a long run.

Linear case

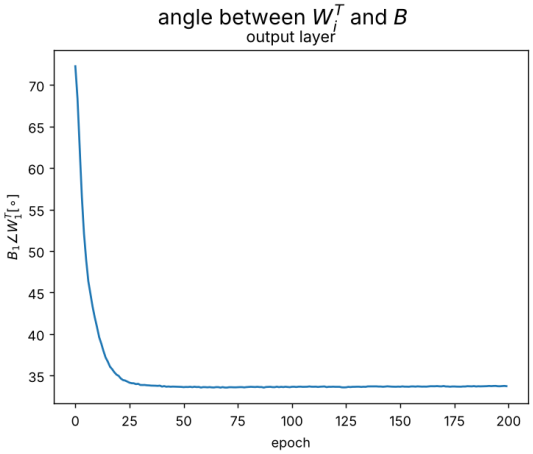


Figure 1: Angle between B and W^T in the linear network.

Why a signal through B can reduce the loss. Let the network be $y = WW_0x$ with hidden layer W_0 and error $\delta = y - t$. The true gradient for W_0 is $W^T\delta\,x^T$, and FA uses the update $\Delta W_0 = -\eta\,B\delta\,x^T$ instead. To first order, the change of the loss from this update is

$$\Delta L = -\eta\left(\delta^T W B\,\delta\right)\|x\|^2.$$

So the loss goes down whenever $\delta^T W B\,\delta > 0$. This is the case if WB is roughly positive, which is what a small angle between B and W^T means (angle below 90°). In this case the FA update is not the gradient, but it has a positive overlap with it, so it still moves downhill.

What the plot shows. At the beginning the angle is about 72°, so B and W^T are almost unrelated. Within roughly 30 epochs it drops to about 34° and then stays there. At the same time the loss goes down (0.054 after 10 epochs, 0.0048 after 200), so the hidden layer does get a useful signal.

Why the angle becomes small. In the beginning only the output layer W gets the exact error. The hidden weights are updated with $B\delta$, so the hidden activity $h = W_0x$ gets a part proportional to $-B\delta$. The update of the output weights is $\Delta W = -\eta\,\delta\,h^T$. Putting in the part

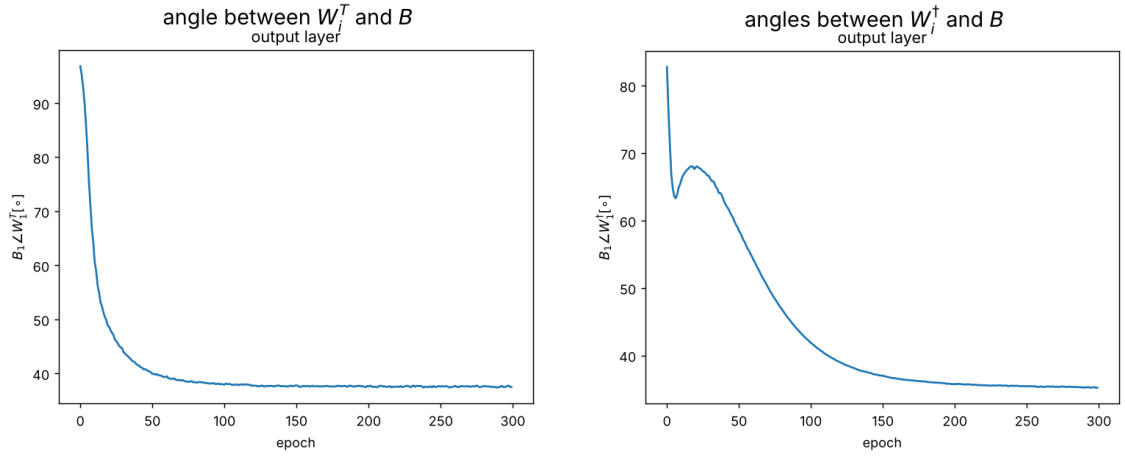
of h that comes from B, this gives a term that looks like $+\eta^2(x \cdot x')\,\delta'\delta^T B^T$ (for an error δ' on a later sample x'). Averaged over the samples, $\delta'\delta^T$ behaves like a positive matrix and $x \cdot x'$ is mostly positive, so this term points roughly along B^T. So W is pushed towards B^T by its own learning rule. The angle stops at 34° and not at 0° for two reasons, I think. First, W has 2 jobs:

1. it has to align with B^T and it has to solve the task
2. when the error gets small, both updates get small, so the alignment stops changing.

[Optional] Does the hidden layer need feedback? I think yes. The whole network is one linear map from 30 inputs to 10 outputs, but a random fixed hidden layer with 20 units only sees 20 random directions of the 30 inputs. I tested this by training only the output layer on a fixed random hidden layer (least squares with bias): the test error was 0.037. FA reaches 0.0077, which is the same as the best linear fit on all 30 inputs. So the hidden layer must learn something, and it can only do this because the feedback is useful.

Nonlinear case

The next question is whether the same thing happens when the hidden layer has a sigmoid. Now the FA signal is $\delta_h = (B\delta_{out}) \odot \sigma'(z)$. The factor $\sigma'(z)$ is positive, so it changes the size of the signal but not the sign of each component. I would therefore expect the same argument to work.



(a) B and W^T (b) B and $W^\dagger$

Figure 2: Angles in the nonlinear network with FA.

The plots confirm this. Both angles start close to 90° (about 97° and 83°) and go down to about 38° and 35°, and the test error is 0.0126. So FA also works with a nonlinearity. The alignment is slower than in the linear case. The two runs also used different learning rates (0.1 and 0.3), so I checked this by running both networks with learning rate 0.1. The angle between B and W^T needs 8 epochs to get below 45° in the linear network and 84 epochs in the nonlinear one, so the nonlinearity itself makes it about 10 times slower. With learning rate 0.3 the nonlinear network needs 29 epochs, so the time scales roughly with 1/lr. This fits the idea that the sigmoid makes the signals smaller ($\sigma' \leq 0.25$), but I did not test the derivative directly.

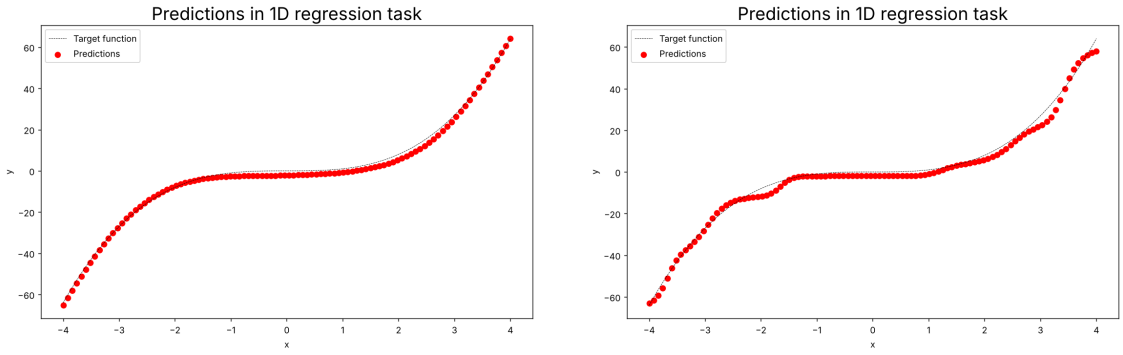
The angle with the pseudo-inverse first goes up a little (to about 68°) before it goes down. I looked at the singular values of W for this. At the beginning the ratio between the largest and the smallest one is large (about 230 in epoch 1 and still about 100 in epoch 17, only about 45 in epoch 31 and about 6 at the end). The pseudo-inverse is dominated by the smallest singular values, which belong to the directions that are not yet aligned. So $W^\dagger$ can only align with B after W has become better conditioned, which is later than for W^T. This matches the plots, but it is only an explanation from one run.

[Optional] Why does W align with B^T and $B^\dagger$ at the same time? W has the same shape as B^T. If $W \approx c\,B^T$, then $W^\dagger \approx B(B^TB)^{-1}/c$. For a random B with 20 rows and 10 columns, B^TB is close to a multiple of the identity, so $W^\dagger$ is roughly a multiple of B. So these two alignments are almost the same thing, and the two plots end at similar angles.

Comparison with BP. In BP the angle is 0° by definition, and the results are a bit better: the test error is 0.0092 compared to 0.0126 for FA, and BP is faster at the start (loss 0.037 after 10 epochs against 0.109). This fits the picture from above. At the beginning B and W^T are almost orthogonal, so the hidden layer learns little until the alignment has developed, while BP has the exact signal from the first step. Even at the end the angle is still about 35°, so the FA signal is only an approximation of the gradient.

Polynomial fitting

In the last experiment I train for many more epochs on a small, noisy dataset (20 points of a cubic with noise), to see what solutions the two methods find. Since FA does not follow the exact gradient, it is interesting whether this makes a difference for overfitting.



(a) Feedback alignment

(b) Backpropagation

Figure 3: Polynomial regression after 20000 epochs.

	training loss	test error
FA	2.77	1.39
BP	1.94	2.63

Both methods learn the cubic shape. BP reaches the lower training loss, but its test error is higher, and its curve has bumps (for example around $x = -2$ and $x = 3.5$): it follows single noisy training points, which is overfitting. The FA curve is smoother and closer to the true cubic, even though its training loss is higher. I repeated the experiment with three other seeds and FA had the lower test error every time (0.94 to 1.74 for FA, 3.27 to 3.47 for BP), so this is not just luck.

A possible explanation connects to the previous sections. In FA the forward weights have to do two things at once: reduce the error and align with B. This restricts how freely the network can fit the noise, so FA acts a bit like a regularizer. I cannot fully prove this, because the two methods use different learning rates in the sheet (0.2 for FA and 0.02 for BP). I tried swapping them, but then neither trained well (BP with 0.2 reached a test error of about 131, FA with 0.02 about 53 after 20000 epochs), so a comparison with the same learning rate was not possible.

[Optional] Possible improvement. The alignment at the beginning is the slow part. One idea is to let B move slowly towards W^T instead of keeping it fixed. This would speed up the alignment, but it would make the feedback less "random" and closer to using the weights themselves.

Summary

FA works because the forward weights learn to align with the random feedback matrix. The error signal through B then has a positive overlap with the real gradient, and the loss goes down. This alignment happens in the linear and in the nonlinear network, but it takes some epochs and is never perfect, which is why FA is a bit slower and slightly worse than BP on the student-teacher task. On the noisy polynomial task the same imperfection seems to help, because FA overfits less than BP.

Implementation

Only `forward()` and `backward()` were filled in. A is the input, W the weights, b the bias, Z the output and δ_X is the gradient of the loss with respect to X.

Linear layer: $Z = AW^T + b,\ \delta_A = \delta_Z W,\ \delta_W = \delta_Z^T A,\ \delta_b = \sum \delta_Z$ over the batch.

```
# forward
Z = A @ W.t()
if b is not None:
    Z = Z + b
# backward
grad_A = grad_Z @ W
grad_W = grad_Z.t() @ A
grad_b = grad_Z.sum(dim=0)
```

Sigmoid: $A = \sigma(Z),\ \delta_Z = \delta_A \odot \sigma(Z)(1 - \sigma(Z))$.

```
# forward
A = torch.sigmoid(Z)
# backward
s = torch.sigmoid(Z)
grad_Z = grad_A * s * (1 - s)
```

MSE loss: $L = \frac{1}{B}\sum_b \frac{1}{2}\|A_b - T_b\|^2,\ \delta_A = \frac{1}{B}(A - T),\ \delta_T = -\frac{1}{B}(A - T)$.

```
# forward
L = 0.5 * ((A - T) ** 2).sum() / A.shape[0]
# backward
grad_A = grad_L * (A - T) / A.shape[0]
grad_T = -grad_L * (A - T) / A.shape[0]
return grad_A, grad_T
```

Linear layer for FA: the forward pass is the same. In the backward pass the error for the previous layer is sent back with B instead of W: $\delta_A = \delta_Z B^T$.

```
# backward
grad_A = grad_Z @ B.t()
grad_W = grad_Z.t() @ A
grad_b = grad_Z.sum(dim=0)
```